PredLDA

Mise en production, intégration DevOps
et livraison du package R

Rapport d'intégration — M2 SISE Programmation R 2026–2027

Thomas Fonferrier

Avec Ryan Ducret, Aya Saadi, Xilian Jing — Équipe G

12 novembre 2026

Résumé

Ce document décrit **tout le travail d'intégration** réalisé pour mettre en production et livrer le package R `PredLDA` : architecture du dépôt GitHub, métadonnées du package, documentation anglaise aux normes R, tutoriel reproductible, jeux d'exemple, application Shiny embarquée, intégration continue, artefacts de livraison (`.tar.gz`, rapports) et procédure de validation d'installation. Il se concentre sur la *chaîne de livraison*, complémentaire au rapport méthodologique LDA.

Table des matières

1 Contexte et objectifs de l'intégration **2**

2 Architecture du dépôt et du package **2**
2.1 Organisation GitHub . . . 2
2.2 Structure du package . . . 3

3 Métadonnées DESCRIPTION et correction Authors@R **3**
3.1 Contenu livré . . . 3
3.2 Incident de production corrigé . . . 3

4 Documentation anglaise aux normes R **3**

5 Tutoriel reproductible et jeux d'exemple **4**

6 Application Shiny embarquée **4**

7 Contrôle qualité : tests, check et CI **4**
7.1 Tests . . . 4
7.2 R CMD check . . . 4
7.3 GitHub Actions . . . 4

8 Chaîne de build et artefacts **5**
8.1 Archive source (plan B) . . . 5
8.2 Installation en ligne . . . 5
8.3 Flux de mise en production . . . 5

9 Procédure de validation « comme le professeur » **5**

10 Inventaire des livrables d'intégration **6**

11 Limites restantes **6**

12 Conclusion **6**

A Commandes rapides **6**

B Trame du mail de rendu **6**

1 Contexte et objectifs de l'intégration

Le cahier des charges impose un package :

— installable **directement depuis GitHub** ;

— documenté en anglais aux normes R (`?LDA` comparable à `?glm`) ;

— accompagné d'un **tutoriel reproductible** et de jeux d'exemple ;

— livré avec une application Shiny, un rapport LaTeX et un plan B hors ligne (archive `.tar.gz`).

Le rôle d'intégration (DevOps léger) consiste à transformer un code algorithmique et une application Shiny en un **produit R livrable** : structure standard, métadonnées valides, documentation, tests, CI et procédure de vérification « comme le destinataire final ».

Notion	Équivalent concret ici
Build	`R CMD build` / `devtools::build()`
Contrôle qualité	`R CMD check` / `devtools::check()`
Distribution	GitHub + `remotes::install_github`
CI	GitHub Actions (`R-CMD-check`)
Artefact plan B	`PredLDA_0.1.0.tar.gz`
Documentation runtime	pages Rd + vignette + README

TABLE 1 – Correspondance DevOps ↔ écosystème package R.

2 Architecture du dépôt et du package

2.1 Organisation GitHub

Le dépôt https://github.com/thomas-fonferrier/projet-lda-r héberge le package dans le sous-dossier `PredLDA/` (et non à la racine). Cette organisation impose le paramètre `subdir = "PredLDA"` lors de l'installation GitHub.

À la racine du dépôt ont été ajoutés :

— `README.md` — point d'entrée (install, quick start, liens) ;

— `.gitignore` — exclusion des artefacts locaux ;

— `.github/workflows/R-CMD-check.yml` — intégration continue.

2.2 Structure du package

```
PredLDA/
|-- DESCRIPTION          # identite, auteurs, dependances
|-- NAMESPACE            # exports (genere par roxygen2)
|-- LICENSE              # MIT
|-- README.md            # tutoriel EN reproductible
|-- R/
|   |-- lda_r6.R         # classe publique LDA + doc roxygen
|   +-- stats_utils.R    # briques statistiques internes
```

```
|-- man/                # aide ?LDA (Rd)
|-- tests/              # tests unitaires / reference MASS
|-- vignettes/          # vignette predlda-intro
|-- inst/
|   |-- extdata/        # jeux d'exemple du tutoriel
|   +-- shiny/          # application Shiny + examples/
|-- data-raw/           # scripts de sync des exemples
+-- report/             # sources LaTeX (exclus du build R)
```

Points d'intégration importants :

— `report/` est listé dans `.Rbuildignore` : le rapport n'est pas embarqué dans le package installé ;

— les placeholders vides `ui.R`/`server.R`/`global.R` ont été supprimés : l'app est entièrement dans `app.R` ;

— `inst/extdata/` expose les mêmes fichiers d'exemple que Shiny via `system.file("extdata", ...)`.

3 Métadonnées DESCRIPTION et correction Authors@R

3.1 Contenu livré

Le fichier `DESCRIPTION` déclare le titre, la version `0.1.0`, la licence MIT, les quatre auteurs de l'équipe G, `Imports: R6`, les `Suggests` (MASS, shiny, readxl, knitr, rmarkdown), `VignetteBuilder: knitr`, ainsi que `URL`/`BugReports`.

3.2 Incident de production corrigé

Une première installation GitHub a échoué avec le message :

```
Authors@R field gives no person with maintainer role,
valid email address and non-empty name.
```

Cause : syntaxe R invalide.

```
# Incorrect : "cre" n'est PAS un role
role = "aut", "cre"

# Correct
role = c("aut", "cre")
```

Sans le rôle `cre` (maintainer), `R CMD build` refuse le package. Correction poussée sur GitHub : Ryan Ducret est `[aut, cre]`.

Leçon DevOps : valider `Authors@R` avant tout push destiné à `install_github`.

4 Documentation anglaise aux normes R

La documentation est écrite en roxygen2 dans `R/lda_r6.R`, puis régénérée par :

```
devtools::document("PredLDA")
```

Cela produit `man/LDA.Rd`, `man/PredLDA-package.Rd` et met à jour le `NAMESPACE`.

La page `?LDA` documente désormais :

— l'API (`new`, `fit`, `predict`, `predict_proba`, `score`, `print`, `summary`) ;

— les paramètres et champs exposés ;

— une section **Reading the results** (une colonne = une classe, intercept en première ligne, softmax, matching par nom, cas binaire $D(x)$) ;

— des exemples exécutables, y compris un fichier `extdata`.

5 Tutoriel reproductible et jeux d'exemple

Trois niveaux :

1. README racine du dépôt — installation GitHub et quick start ;
2. `PredLDA/README.md` — tutoriel EN complet ;
3. vignette `predlda-intro` — `vignette("predlda-intro", package = "PredLDA")`.

Fichiers livrés dans `inst/extdata/` : `iris_apprentissage.tsv`, `iris_apprentissage_fr.csv`, `iris_nouveaux.tsv`.

```
ex <- system.file("extdata", "iris_nouveaux.tsv", package = "PredLDA")
newdata <- read.delim(ex, check.names = FALSE)
```

6 Application Shiny embarquée

L'application est livrée sous `inst/shiny/` :

```
shiny::runApp(system.file("shiny", package = "PredLDA"))
```

Le tutoriel Shiny EN détaille un parcours reproductible (iris, split 80/20, prédiction individu/fichier). Les dépendances Shiny sont en `Suggests`.

7 Contrôle qualité : tests, check et CI

7.1 Tests

— `tests/test-LDA.R` — API, validations, accord `MASS::lda` ;

— `tests/test-reference.R` — formules textbook + hors-échantillon ;

— `inst/shiny/tests/test-server.R` — logique serveur Shiny.

7.2 R CMD check

```
devtools::check("PredLDA", args = c("--no-manual"))
# Status: OK
```

7.3 GitHub Actions

Le workflow `.github/workflows/R-CMD-check.yml` se déclenche à chaque `push`/`pull_request` sur `main`, avec `working-directory: PredLDA`, pour détecter toute régression avant livraison.

8 Chaîne de build et artefacts

8.1 Archive source (plan B)

```
devtools::build("PredLDA")
# -> PredLDA_0.1.0.tar.gz

install.packages("PredLDA_0.1.0.tar.gz",
                 repos = NULL, type = "source")
```

8.2 Installation en ligne

```
remotes::install_github(
  "thomas-fonferrier/projet-lda-r",
  subdir = "PredLDA",
  upgrade = "never"
)
```

8.3 Flux de mise en production

1. Intégrer le code dans `PredLDA/` ;
2. `devtools::document()` puis tests locaux ;
3. `devtools::check()` jusqu'à `Status: OK` ;
4. commit + push sur `main` ;
5. vérifier GitHub Actions ;
6. tester `install_github` en session R propre ;
7. reconstruire `.tar.gz` et PDF ;
8. déposer sur le drive + mail au responsable.

9 Procédure de validation « comme le professeur »

Dans RStudio, sans dépendre du dossier local :

```
try(remove.packages("PredLDA"), silent = TRUE)
remotes::install_github(
  "thomas-fonferrier/projet-lda-r",
  subdir = "PredLDA",
  upgrade = "never",
  force = TRUE
)
library(PredLDA)
?LDA
vignette("predlda-intro", package = "PredLDA")

model <- LDA$new()
model$fit(iris[, 1:4], iris$Species)
model$predict(iris[c(1, 51, 101), 1:4])

shiny::runApp(system.file("shiny", package = "PredLDA"))
```

Checklist : installation OK, `?LDA` lisible, vignette accessible, `extdata` non vide, Shiny démarre depuis le package installé.

10 Inventaire des livrables d'intégration

11 Limites restantes

Hors périmètre de cette intégration (équipe algo / suite) : prétraitement des variables catégorielles, sélection automatique de variables, indicateurs de pertinence, enrichissement éventuel du rapport méthodologique (captures Shiny, résultats détaillés).

Livrable	Implémentation réalisée
Install GitHub	README + `subdir` + DESCRIPTION valide
Maintainer	Correction `Authors@R` (`c("aut","cre")`)
Aide EN	Roxygen enrichi + `man/*.Rd`
Tutoriel EN	README package + vignette `predlda-intro`
Jeux d'exemple	`inst/extdata/` + sync `data-raw/`
Shiny livrable	`inst/shiny/app.R`, via `system.file`
CI	GitHub Actions R-CMD-check
Plan B	`PredLDA_0.1.0.tar.gz` buildé et testé
Qualité	`R CMD check` : Status OK
Rapports	`rapport.tex` + `rapport_integration.tex`

TABLE 2 – Synthèse des livrables d'intégration.

12 Conclusion

La mise en production de PredLDA repose sur une chaîne complète : dépôt GitHub structuré, métadonnées valides, documentation et tutoriel anglais, exemples embarqués, check R + CI, archive hors ligne et test d'installation en session propre. Ces éléments permettent de **livrer** le package conformément au cahier des charges.

Dépôt : https://github.com/thomas-fonferrier/projet-lda-r

A Commandes rapides

```
# Qualite locale
Rscript -e 'devtools::document("PredLDA")'
Rscript -e 'devtools::check("PredLDA", args="--no-manual")'
Rscript -e 'devtools::build("PredLDA", path=".")'

# Publication
git add -A
git commit -m "Release integration docs"
git push
```

B Trame du mail de rendu

```
A : ricco.rakotomalala@univ-lyon2.fr
Objet : [SISE - Prog. R - Equipe G] Fonferrier Ducret Saadi Jing

Bonjour,

Lien du projet PredLDA :
https://github.com/thomas-fonferrier/projet-lda-r

Livrables : package installable (subdir PredLDA), application Shiny,
tutoriel EN, archive PredLDA_0.1.0.tar.gz, rapports PDF/LaTeX.

Cordialement,
Equipe G
```